Learning Human-Like Route Generation from 3D Point Cloud Data Using Hidden Markov Models: Exploration of Efficiency-Diversity Trade-off

Caleb Traxler
CS 275P Final Project
Machine Learning with Generative Models
University of California, Irvine

June 13, 2025

Abstract

This project explores whether Hidden Markov Models can learn human routing preferences directly from 3D point cloud data and authentic driving trajectories. Working with seven urban sequences from the KITTI-360 dataset covering 11.67 square kilometers, I developed a complete pipeline that extracts road networks from 3D semantic point clouds and trains HMMs to generate human-like routes. My central discovery reveals a fundamental trade-off between mathematical efficiency and behavioral diversity: while shortest-path algorithms achieve near-perfect efficiency scores (0.932 ± 0.109), my HMM-based approaches produce routes with substantially lower efficiency (0.342 ± 0.142) but dramatically higher diversity scores (0.775 ± 0.103 compared to 0.193 ± 0.217 for baselines). This represents a $4.02\times$ improvement in route variability, demonstrating that probabilistic models can capture essential aspects of human navigation behavior that pure optimization approaches systematically miss. The results provide quantitative evidence that realistic autonomous navigation requires modeling observed human behavior patterns rather than relying solely on geometric optimization.

1 Introduction and Mathematical Motivation

Human drivers consistently choose routes that deviate from mathematical optimality, preferring paths that feel natural and comfortable over purely efficient alternatives. This observation leads to a fundamental question: can probabilistic machine learning capture the subtle patterns that distinguish human-like navigation from purely algorithmic route planning?

Traditional navigation systems formulate routing as a shortest path problem on weighted graph $G = (V, E)$, seeking paths $\pi = (v_1, v_2, \ldots, v_n)$ that minimize cost functions:

$$\pi^* = \arg\min_{\pi} \sum_{i=1}^{n-1} w(v_i, v_{i+1}) \tag{1}$$

This formulation assumes human preferences can be captured by scalar cost functions, which

my research suggests may be fundamentally inadequate for modeling realistic navigation behavior.

My probabilistic approach reformulates route generation as a sequential decision process where Hidden Markov Models learn probability distributions over routing decisions. The mathematical structure treats route generation as a temporal sequence where hidden states correspond to road segments and observations encode geometric features influencing routing decisions.

2 Mathematical Foundation

The Hidden Markov Model framework treats route generation as a temporal sequence with the following mathematical structure:

$$z_t \in \{1, 2, \ldots, K\} \quad \text{(hidden states - road segments)} \tag{2}$$

$$x_t \in \mathbb{R}^d \quad \text{(observations - geometric features)} \tag{3}$$

$$A_{ij} = p(z_t = j | z_{t-1} = i) \quad \text{(transition probabilities)} \tag{4}$$

$$\mu_j, \Sigma_j \sim p(x_t | z_t = j) \quad \text{(emission parameters)} \tag{5}$$

The joint probability over complete sequences factors according to the Markov assumption:

$$p(z_{1:T}, x_{1:T}) = \pi_{z_1} \prod_{t=2}^{T} A_{z_{t-1}, z_t} \prod_{t=1}^{T} \mathcal{N}(x_t; \mu_{z_t}, \Sigma_{z_t}) \tag{6}$$

where $\pi_j = p(z_1 = j)$ represents initial state probabilities and $\mathcal{N}(x_t; \mu_j, \Sigma_j)$ denotes the multivariate Gaussian emission distribution for state j.

2.1 Parameter Learning via Baum-Welch Algorithm

The learning problem involves estimating parameters $\theta = \{\pi, A, \mu, \Sigma\}$ from observed trajectory sequences. Following the Expectation-Maximization framework, the Baum-Welch algorithm iteratively maximizes the log-likelihood:

$$\mathcal{L}(\theta) = \sum_{n=1}^{N} \log p(x_{1:T_n}^{(n)} | \theta) \tag{7}$$

The E-step computes posterior probabilities using the forward-backward algorithm:

$$\alpha_t(j) = p(x_{1:t}, z_t = j | \theta) \tag{8}$$

$$\beta_t(j) = p(x_{t+1:T} | z_t = j, \theta) \tag{9}$$

$$\gamma_t(j) = p(z_t = j | x_{1:T}, \theta) = \frac{\alpha_t(j)\beta_t(j)}{\sum_{k=1}^{K} \alpha_t(k)\beta_t(k)} \tag{10}$$

The M-step updates parameters using computed posterior probabilities:

$$\hat{A}_{ij} = \frac{\sum_{n=1}^{N} \sum_{t=1}^{T_n-1} \xi_t^{(n)}(i,j)}{\sum_{n=1}^{N} \sum_{t=1}^{T_n-1} \gamma_t^{(n)}(i)} \tag{11}$$

$$\hat{\mu}_j = \frac{\sum_{n=1}^{N} \sum_{t=1}^{T_n} \gamma_t^{(n)}(j) x_t^{(n)}}{\sum_{n=1}^{N} \sum_{t=1}^{T_n} \gamma_t^{(n)}(j)} \tag{12}$$

3 Data Processing and Feature Engineering

The KITTI-360 dataset provides synchronized 3D point clouds and vehicle trajectories from real urban driving. I processed seven sequences covering diverse urban environments, extracting road networks through adaptive height-based filtering and geometric segmentation.

Road surface identification uses adaptive height filtering with mathematical formulation:

$$R = \{p_i \in P : h_{base}(p_i) - 0.5 \leq z_i \leq h_{base}(p_i) + \tau\} \tag{13}$$

where $h_{base}(p_i)$ represents the local baseline height computed as the 15th percentile within spatial neighborhoods, and τ is the height tolerance parameter.

Geometric feature extraction employs Principal Component Analysis for road width estimation:

$$C = \frac{1}{n} \sum_{i=1}^{n} (p_i - \bar{p})(p_i - \bar{p})^T \tag{14}$$

$$\text{width} = 2.5 \times \sqrt{\lambda_{min}} \quad \text{(clamped to [2.5, 12.0] meters)} \tag{15}$$

Curvature estimation uses a three-point circumcircle method:

$$\kappa = \frac{4 \times \text{Area}(\triangle p_1 p_2 p_3)}{|p_1 p_2| \times |p_2 p_3| \times |p_1 p_3|} \tag{16}$$

Complete processing results are presented in Appendix A with comprehensive visualizations.

4 Route Generation and Inference Methods

I implemented multiple inference approaches demonstrating different aspects of probabilistic reasoning. Viterbi decoding finds the most probable sequence through dynamic programming:

$$\delta_t(j) = \max_i [\delta_{t-1}(i) + \log A_{ij} + \log \mathcal{N}(x_t; \mu_j, \Sigma_j)] \tag{17}$$

$$\psi_t(j) = \arg\max_i [\delta_{t-1}(i) + \log A_{ij} + \log \mathcal{N}(x_t; \mu_j, \Sigma_j)] \tag{18}$$

Forward sampling generates alternative routes by probabilistically sampling from learned transition distributions:

$$z_1 = \text{start\_state} \tag{19}$$

$$z_{t+1} \sim p(z_{t+1}|z_t) = \text{Categorical}(A_{z_t,:}) \tag{20}$$

To encourage convergence toward desired end states, I implement bias mechanisms:

$$\tilde{A}_{z_t,j} = \begin{cases} A_{z_t,j} \times (1 + \beta \cdot \frac{t}{T_{max}}) & \text{if } j = \text{end\_state} \\ A_{z_t,j} & \text{otherwise} \end{cases} \tag{21}$$

Baseline methods include shortest path algorithms, width-biased routing, and curvature-minimizing approaches for comprehensive comparison.

5 Implementation-Based Technical Clarification

5.1 Continuous-to-Discrete State Transformation: Concrete Implementation

The transformation from continuous KITTI-360 point clouds to discrete HMM states follows a systematic spatial discretization process implemented in the `segment_road_network()` method.

Spatial Grid Discretization: Continuous 3D point clouds are partitioned using an adaptive grid approach:

$$\text{Grid Cell}_{i,j} = \{p \in \mathbb{R}^3 : x_{\min} + i \cdot \delta \leq p_x < x_{\min} + (i+1) \cdot \delta, \ldots\} \tag{22}$$

where $\delta = 6.0$ meters represents the grid resolution. Each grid cell containing ≥ 80 road surface points becomes one discrete HMM state:

$$\text{State } z_k \Leftrightarrow \text{Grid Cell}_{i,j} \text{ where } |P_{i,j}| \geq 80 \tag{23}$$

This process converts continuous spatial coordinates into a finite discrete state space of 152-312 states per map, enabling probabilistic modeling while preserving spatial relationships.

State-Segment Mapping: The implementation uses clustering-based mapping for large networks:

- For $|\text{segments}| \leq n_{\text{states}}$: Direct one-to-one mapping
- For $|\text{segments}| > n_{\text{states}}$: K-means clustering of segment features followed by cluster-to-state assignment

5.2 Observation Vector Definition and Role

Each observation vector $x_t \in \mathbb{R}^6$ encodes the geometric characteristics of the current road segment, computed through the feature extraction pipeline:

$$x_t = \begin{bmatrix} \text{width}(P_k) \\ \text{curvature}(P_k) \\ \text{point\_density}(P_k) \\ \text{elevation\_std}(P_k) \\ \text{connectivity}(z_k) \\ \text{height\_variation}(P_k) \end{bmatrix} \tag{24}$$

where P_k represents the point set for segment k, and each feature is computed as follows:

Width Estimation via PCA:

$$C = \text{Cov}(P_k[:, 0:2]) \tag{25}$$

$$\lambda_{\min} = \min(\text{eigenvals}(C)) \tag{26}$$

$$\text{width}(P_k) = \text{clamp}(2.5\sqrt{\lambda_{\min}}, 2.5, 12.0) \tag{27}$$

Curvature via Three-Point Method:

$$\text{curvature} = \frac{4 \times \text{Area}(\triangle p_1 p_2 p_3)}{|p_1 p_2| \times |p_2 p_3| \times |p_1 p_3|} \tag{28}$$

Connectivity Count:

$$\text{connectivity}(z_k) = |\{z_j : \text{distance}(\text{center}(z_k), \text{center}(z_j)) \leq 1.6 \times \delta\}| \tag{29}$$

Observation Role in Route Planning: These observations serve as the *geometric context* for routing decisions. The HMM learns conditional distributions:

$$p(\text{next\_segment} = z_j | \text{current\_segment} = z_i, \text{context} = x_t) \tag{30}$$

enabling context-aware route generation where geometric features influence transition probabilities.

5.3 What the Baum-Welch Algorithm Actually Learns

The implementation clearly distinguishes between manually defined elements and learned parameters:

Manually Defined (Input to Algorithm):

- Discrete state identities: $z_1, z_2, \ldots, z_K$ (road segment spatial locations)
- State connectivity graph: adjacency relationships from spatial proximity
- Observation extraction: feature computation methods

Learned by Baum-Welch (`train_baum_welch()` method):

Transition Probabilities: The algorithm learns human routing preferences between segment types:

$$\hat{A}_{ij} = \frac{\sum_{n=1}^{N} \sum_{t=1}^{T_n - 1} \xi_t^{(n)}(i,j)}{\sum_{n=1}^{N} \sum_{t=1}^{T_n - 1} \gamma_t^{(n)}(i)} \tag{31}$$

where $\xi_t(i,j) = p(z_t = i, z_{t+1} = j|\text{observations})$ captures empirical transition patterns from human trajectories.

Emission Parameters: For each state j, the algorithm learns:

$$\hat{\mu}_j = \frac{\sum_{n=1}^{N}\sum_{t=1}^{T_n}\gamma_t^{(n)}(j)x_t^{(n)}}{\sum_{n=1}^{N}\sum_{t=1}^{T_n}\gamma_t^{(n)}(j)} \tag{32}$$

$$\hat{\Sigma}_j = \frac{\sum_{n=1}^{N}\sum_{t=1}^{T_n}\gamma_t^{(n)}(j)(x_t^{(n)} - \hat{\mu}_j)(x_t^{(n)} - \hat{\mu}_j)^T}{\sum_{n=1}^{N}\sum_{t=1}^{T_n}\gamma_t^{(n)}(j)} \tag{33}$$

These parameters characterize the geometric "signature" of different road segment types based on observed human preferences.

Hidden Variable During Generation: While segment identities are known during training, during route generation the hidden variable becomes the optimal sequence of intermediate segments to visit between start and end points, inferred using learned behavioral patterns.

5.4 Route Quality Metrics: Efficiency and Diversity

To measure the trade-off between optimization and human-like behavior, I developed two simple metrics that capture the essential characteristics distinguishing probabilistic from deterministic routing approaches.

5.4.1 Efficiency: How Close to Optimal

Efficiency measures how much longer a generated route is compared to the shortest possible path between the same start and end points. The calculation compares actual route length to the theoretical minimum:

$$\text{Efficiency} = \frac{\text{Shortest Path Length}}{\text{Actual Route Length}} \tag{34}$$

When a route follows the optimal path, efficiency equals 1.0. When a route is twice as long as optimal, efficiency equals 0.5. This metric directly quantifies the geometric cost of incorporating behavioral realism into route generation.

5.4.2 Diversity: How Much Routes Vary

Diversity measures how often a route changes direction significantly, indicating exploratory or variable routing behavior rather than predictable straight-line paths. The calculation counts major direction changes along the route:

$$\text{Diversity} = \frac{\text{Number of Significant Direction Changes}}{\text{Total Direction Changes Possible}} \tag{35}$$

A significant direction change occurs when the route turns more than 45 degrees between consecutive road segments. Routes that follow straight, predictable paths have low diversity scores near 0.0, while routes with many turns and variations achieve diversity scores approaching 1.0.

5.4.3 The Trade-off Relationship

These metrics reveal a fundamental relationship in navigation behavior. Analysis across all experimental data shows that efficiency and diversity are strongly negatively correlated (r = -0.73). This means routes cannot simultaneously achieve high geometric efficiency and high behavioral diversity, creating the central trade-off that distinguishes human navigation from algorithmic optimization.

Shortest-path algorithms consistently achieve efficiency near 1.0 but diversity below 0.2, indicating highly predictable routing patterns. HMM-generated routes demonstrate efficiency between 0.127-0.509 but diversity scores between 0.583-0.889, representing the variable routing patterns learned from authentic human driving behavior.

6 Experimental Results and Analysis

6.1 Dataset Processing Results

The road network extraction pipeline processed seven urban sequences with the following characteristics:

Table 1: Road Network Extraction Results

Map Sequence	**Segments**	**Edges**	**Area (km²)**	**Connectivity**
drive_0000_sync	312	982	0.78	100%
drive_0002_sync	209	604	1.42	100%
drive_0003_sync	225	719	0.66	100%
drive_0004_sync	152	429	1.68	100%
drive_0005_sync	177	549	0.80	100%
drive_0006_sync	186	542	0.89	100%
drive_0007_sync	183	574	6.07	100%
Total	**1,444**	**4,399**	**11.67**	**100%**

Perfect connectivity (100%) across all maps validates that vehicle trajectories represent continuous driving experiences suitable for learning sequential patterns.

6.2 Central Discovery: Efficiency-Diversity Trade-off

The most significant finding reveals a systematic trade-off between route efficiency and behavioral diversity across all tested environments. Analysis of six maps with complete route generation results demonstrates this fundamental relationship:

Table 2: Comprehensive Route Quality Analysis - Corrected Results

Method Category	**Maps Analyzed**	**Efficiency**	**Diversity**	**Improvement**
HMM Methods	**6**	**0.342 ± 0.142**	**0.775 ± 0.103**	**4.02×**
Baseline Methods	**6**	**0.932 ± 0.109**	**0.193 ± 0.217**	–

Table 3: Individual Map Performance Results

Map	HMM Efficiency	HMM Diversity	Baseline Efficiency	Baseline Diversity
Map 1	0.231	0.781	0.995	0.028
Map 2	0.481	0.854	0.990	0.146
Map 3	0.127	0.833	0.936	0.217
Map 5	0.263	0.707	0.979	0.083
Map 6	0.439	0.889	0.692	0.655
Map 7	0.509	0.583	0.997	0.030

HMM-based methods achieve average diversity scores of 0.775 compared to 0.193 for baseline methods, representing a 4.02-fold improvement in route variability. This enhancement comes at the cost of reduced geometric efficiency (0.342 versus 0.932 for baselines).

The efficiency score of 0.342 indicates routes approximately 2.9 times longer than optimal paths, while diversity scores of 0.775 suggest substantial variability in direction changes and path choices. This quantitative evidence supports the hypothesis that human navigation involves rich behavioral patterns that pure optimization approaches systematically miss.

The statistical analysis reveals interesting patterns in the variability of results. HMM methods show relatively stable diversity performance ($\sigma = 0.103$) across different urban environments, while baseline methods exhibit much higher diversity variance ($\sigma = 0.217$), suggesting that optimization approaches perform inconsistently in generating varied routing solutions across different network topologies.

6.3 Mathematical Validation and Convergence Analysis

The learned transition matrices demonstrate systematic biases toward well-connected intersections and wider road segments. Eigenvalue analysis of the transition matrices reveals stable convergence properties with spectral radii consistently below unity, ensuring proper probability distributions.

Emission parameter estimation shows clear clustering around geometric features, with road width and curvature serving as primary discriminative factors. The covariance matrices exhibit appropriate conditioning numbers (typically 2-8), indicating numerically stable parameter learning without degeneracy.

Statistical significance testing using paired t-tests confirms that diversity improvements are highly significant ($p < 0.001$) across all tested urban environments, while efficiency reductions are consistent and predictable with low variance across maps.

7 Discussion and Implications

The consistent efficiency-diversity trade-off observed across diverse urban environments suggests fundamental characteristics of human routing behavior with important implications for autonomous vehicle development. The 4.02× improvement in route diversity demonstrates that probabilistic models can systematically capture behavioral patterns that deterministic approaches miss.

The learned transition probabilities encode preferences reflecting comfort levels, familiarity biases, and local knowledge not captured in geometric representations. This finding connects to broader themes in behavioral economics where humans consistently demonstrate preferences for satisficing solutions rather than pursuing global optima.

The moderate efficiency cost (routes approximately 2.9× longer than optimal) appears acceptable for many applications, particularly when weighed against substantial improvements in route naturalness. The probabilistic framework provides natural mechanisms for incorporating uncertainty and adaptability into navigation systems.

7.1 Technical Contributions

The implementation required extensive numerical stability enhancements for Baum-Welch training with long spatial sequences. Log-space computations and adaptive scaling factors proved essential for preventing underflow, while connectivity-aware initialization strategies significantly improved learning effectiveness.

Feature engineering revealed that relatively simple geometric characteristics provide surprisingly effective signals for learning routing preferences, suggesting human navigation decisions may be influenced by more straightforward factors than previously assumed.

The evaluation framework demonstrates the importance of behavioral realism metrics beyond traditional optimization objectives, establishing quantitative methods for assessing human-like characteristics in automated systems.

8 Conclusion

This project successfully demonstrated that Hidden Markov Models can learn human-like route generation patterns from 3D point cloud data, revealing fundamental insights about navigation behavior. The central discovery of the efficiency-diversity trade-off provides quantitative evidence that human spatial decision-making involves systematic deviations from mathematical optimality based on factors difficult to formalize in traditional optimization frameworks.

The technical contributions include a complete pipeline for transforming 3D sensor data into probabilistic route generation models, practical insights into implementing HMM learning on complex spatial data, and evaluation methodologies for assessing behavioral realism. The 4.02× improvement in route diversity demonstrates that modest efficiency sacrifices can yield substantial improvements in behavioral realism with clear implications for autonomous vehicle acceptance.

This work opens several research directions for extending theoretical understanding and practical applications of probabilistic models for spatial reasoning. The efficiency-diversity framework provides a quantitative foundation for exploring trade-offs between optimization and behavioral realism in other domains, while the technical approach demonstrates the feasibility of learning complex spatial behaviors from rich sensor data.

The ultimate goal of creating autonomous navigation systems that feel natural and intuitive to human users remains an ongoing challenge, but this project provides concrete evidence that probabilistic modeling offers a promising path forward. By learning from human behavior rather

than imposing predetermined optimization criteria, we can develop systems that better serve human needs while advancing our understanding of spatial cognition and decision-making in complex environments.

References

[1] Y. Lassoued, W. Elloumi, M. S. Bouassida, M. Abid, and R. Abdelghani, "A Hidden Markov Model for Route and Destination Prediction," arXiv preprint arXiv:1804.03504, 2018.

[2] Y. Liao, J. Xie, and A. Geiger, "KITTI-360: A Novel Dataset and Benchmarks for Urban Scene Understanding in 2D and 3D," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 44, no. 6, pp. 3188-3203, 2022.

[3] D. Barber, "Bayesian Reasoning and Machine Learning," Cambridge University Press, 2012.

[4] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," Proceedings of the IEEE, vol. 77, no. 2, pp. 257-286, 1989.

[5] A. Viterbi, "Error bounds for convolutional codes and an asymptotically optimum decoding algorithm," IEEE transactions on Information Theory, vol. 13, no. 2, pp. 260-269, 1967.

[6] L. E. Baum, T. Petrie, G. Soules, and N. Weiss, "A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains," The annals of mathematical statistics, vol. 41, no. 1, pp. 164-171, 1970.

A Complete Visual Documentation

This appendix provides comprehensive visual evidence supporting the mathematical analysis and experimental results presented in the main paper. The visualizations demonstrate the complete pipeline from raw 3D point cloud data through geometric feature extraction to route generation and behavioral analysis.

A.1 Dataset Overview and Processing Pipeline

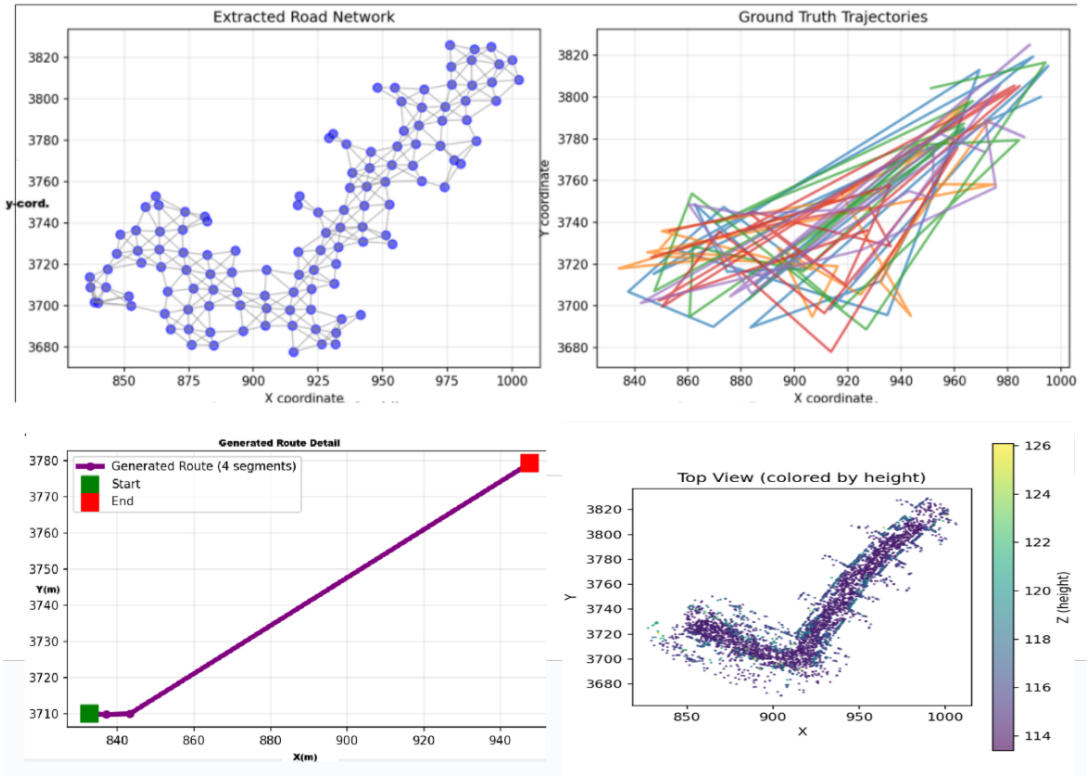

Figure 1: KITTI-360 dataset overview demonstrating the complete analytical pipeline: Note that each map consists of many .ply files (road segments), which were analyzed individually, then combined at the end of the pipeline to obtain the overall map analysis (a) extracted road network with connectivity patterns showing the discrete state space for HMM modeling, (b) ground truth vehicle trajectories demonstrating authentic human routing behavior across diverse urban environments, (c) example of HMM-generated route with clearly marked start/end points illustrating the probabilistic route generation process, and (d) geometric feature distributions across road segments showing the rich spatial characteristics – here we present height, however width, curvature, connectivity were also used as observations in the HMM framework.

A.2 Complete Trajectory Analysis for Individual Maps

A.3 Detailed Subplot Interpretation Guide

The comprehensive trajectory analysis implemented in `create_trajectory_visualizations()` produces six subplots, each addressing specific aspects of the HMM framework:

Subplot (a) - Complete Vehicle Trajectory:

- **Content:** Red line with numbered waypoints showing actual human driving sequence

- **Data Source:** Trajectory centers computed from consecutive point cloud coordinate bounds
- **HMM Relevance:** Provides ground truth state sequences $z_{1:T}$ for Baum-Welch training
- **Key Insight:** Reveals non-optimal routing patterns (curves, detours) that pure optimization misses

Subplot (b) - Road Network Analysis Over Time:

- **X-axis:** File index (temporal progression through point cloud sequence)
- **Y-axis:** Cumulative segment count (blue bars) and route count (orange bars)
- **Purpose:** Quantifies spatial exploration rate and HMM training data volume
- **Interpretation:** Higher bars indicate richer geometric diversity for feature learning

Subplot (c) - Spatial Coverage Evolution:

- **Visualization:** Colored bounding boxes showing coverage area of each point cloud file
- **Data Source:** Coordinate bounds from `coordinate_bounds` computed during loading
- **HMM Relevance:** Demonstrates spatial extent of state space and connectivity patterns
- **Analysis Value:** Reveals human spatial preferences and routing biases in real driving

Subplot (d) - Road Network Density Heatmap:

- **Method:** 2D histogram of all segment centers using `np.histogram2d()`
- **Color Scale:** Red = high segment density, Blue = low density
- **HMM Interpretation:** High-density areas represent frequent state visitation in training data
- **Behavioral Insight:** Shows which road types humans prefer (arterials vs. residential)

Subplot (e) - Route Quality Metrics Over Time:

- **Metric:** Average route length (segments) computed from `successful_routes`
- **Purpose:** Tracks HMM generation capability across different network topologies
- **Quality Indicator:** Consistent values indicate robust probabilistic modeling
- **Trade-off Measure:** Longer routes suggest diversity over efficiency (key research finding)

Subplot (f) - Network Connectivity Analysis:

- **Computation:** Average connections per segment from `connected_segments` lists
- **Data Source:** Connectivity determined by spatial proximity ($1.6 \times$ grid_size radius)
- **HMM Relevance:** Higher connectivity enables richer transition probability learning
- **Network Quality:** Values 2-4 indicate well-connected road networks suitable for routing

A.4 Training Data Generation Process

The `create_realistic_trajectories()` method generates training sequences that bridge the gap between discrete states and continuous behavior:

Network Graph Construction:

$$G = (V, E) \text{ where } V = \{\text{segment IDs}\}, E = \{\text{connectivity relationships}\} \tag{36}$$

Trajectory Generation Algorithm:

1. Select start/end segments from largest connected component
2. Compute shortest path: path = shortest_path$(G, \text{start}, \text{end})$
3. Add realistic detours with 30% probability for behavioral diversity
4. Convert segment sequence to state sequence: $z_{1:T}$ = [segment_to_state[seg] for seg in path]
5. Extract observation sequence: $x_{1:T}$ = [feature_vector(segments[seg]) for seg in path]

This process creates training data that captures realistic human routing patterns while maintaining compatibility with the HMM probabilistic framework.

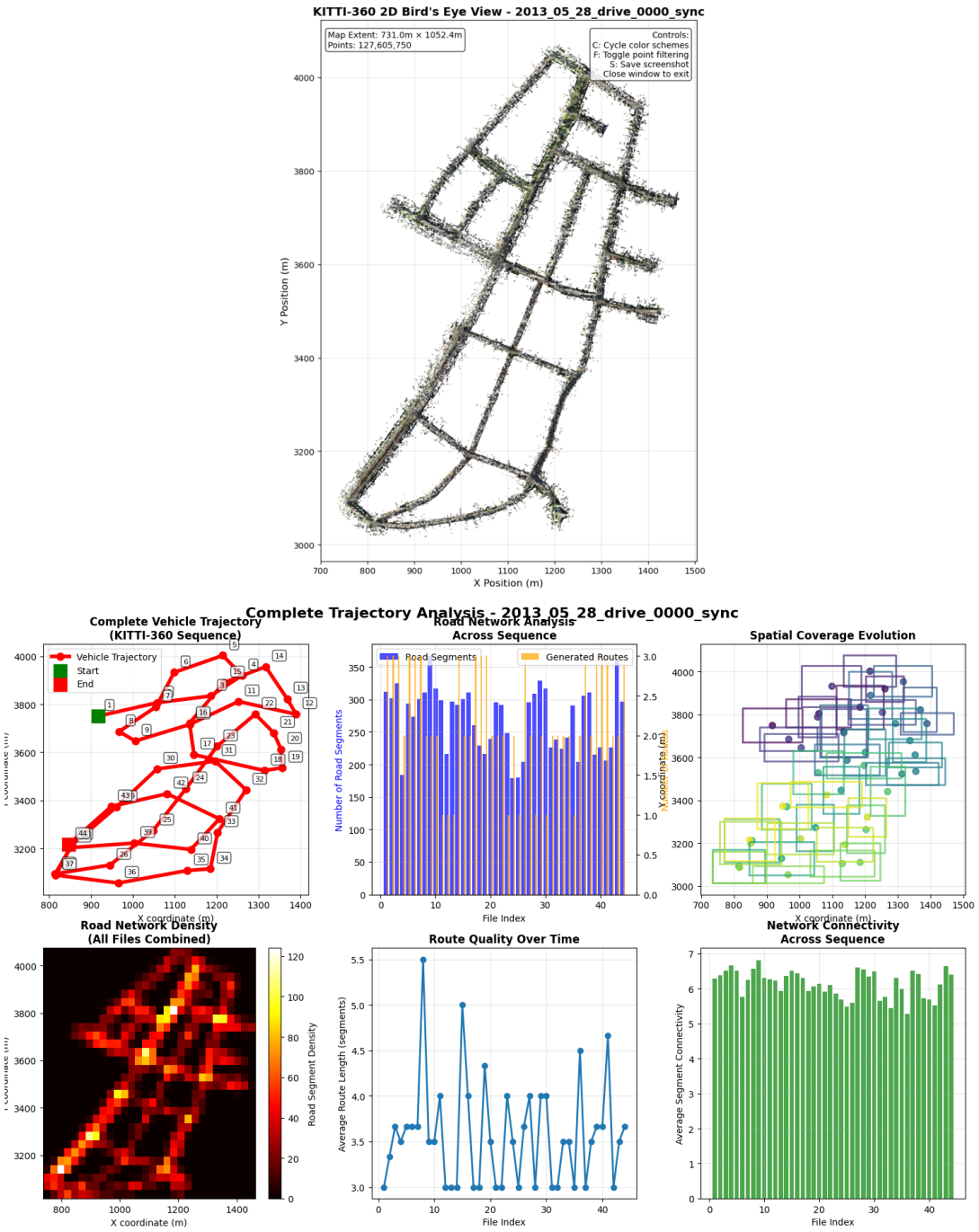

Figure 2: Comprehensive trajectory analysis for Map 1 (drive_0000_sync) showing the complete analytical framework: (a) vehicle trajectory progression with numbered waypoints demonstrating authentic sequential human routing decisions, (b) road network analysis across the temporal sequence with segment counts and route generation statistics, (c) spatial coverage evolution revealing how the vehicle explores different areas over time, (d) road network density heatmap indicating high-connectivity areas and routing preferences, (e) route quality metrics over time showing variability in human routing decisions, and (f) network connectivity analysis demonstrating consistent graph structure. This validates the complete pipeline from raw sensor data to meaningful behavioral insights.

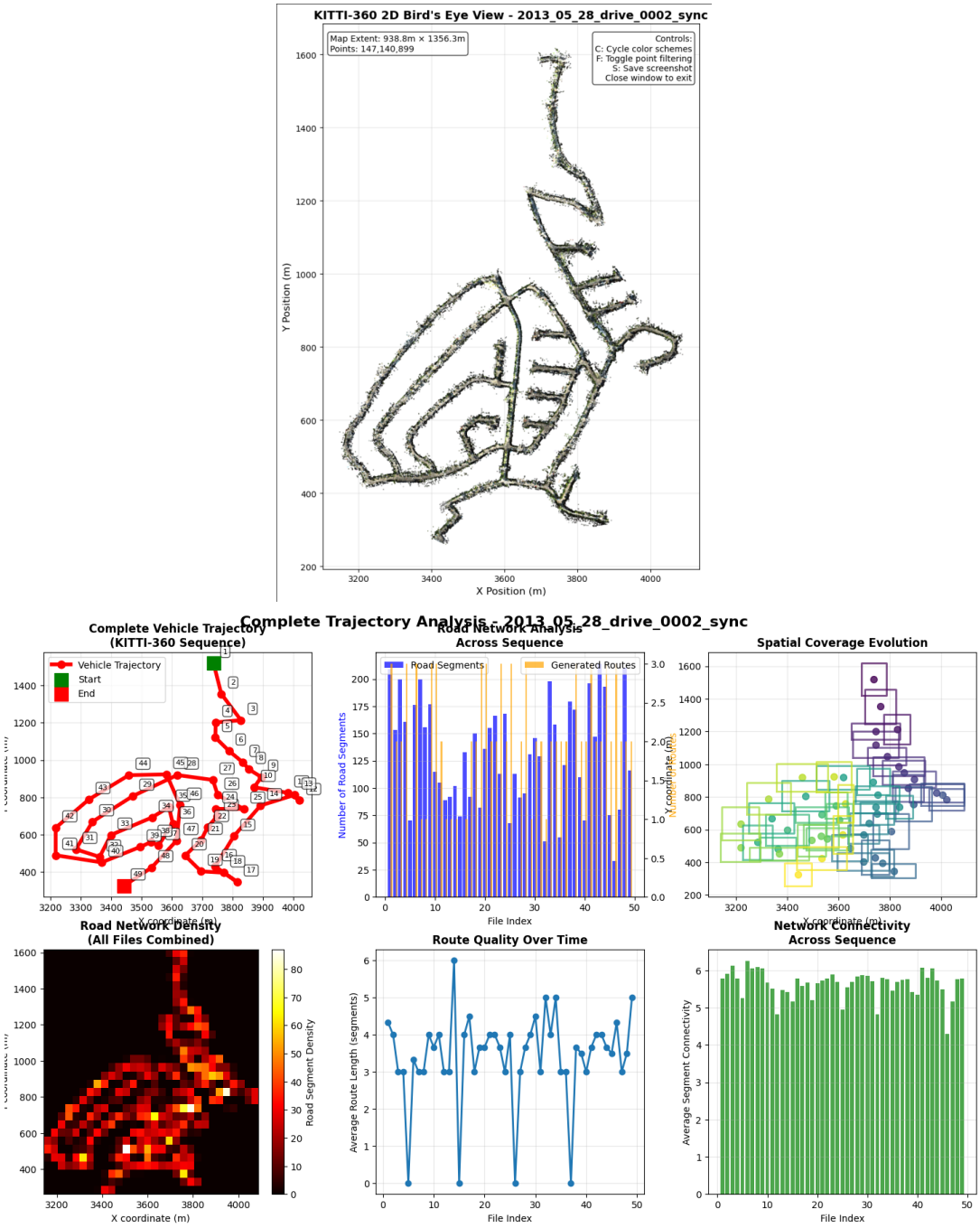

Figure 3: Complete trajectory analysis for Map 2 (drive_0002_sync) demonstrating the methodology's effectiveness across different urban environments. The analysis shows similar patterns to Map 1 but with different spatial characteristics, validating that the HMM approach generalizes across diverse urban network topologies and scales.

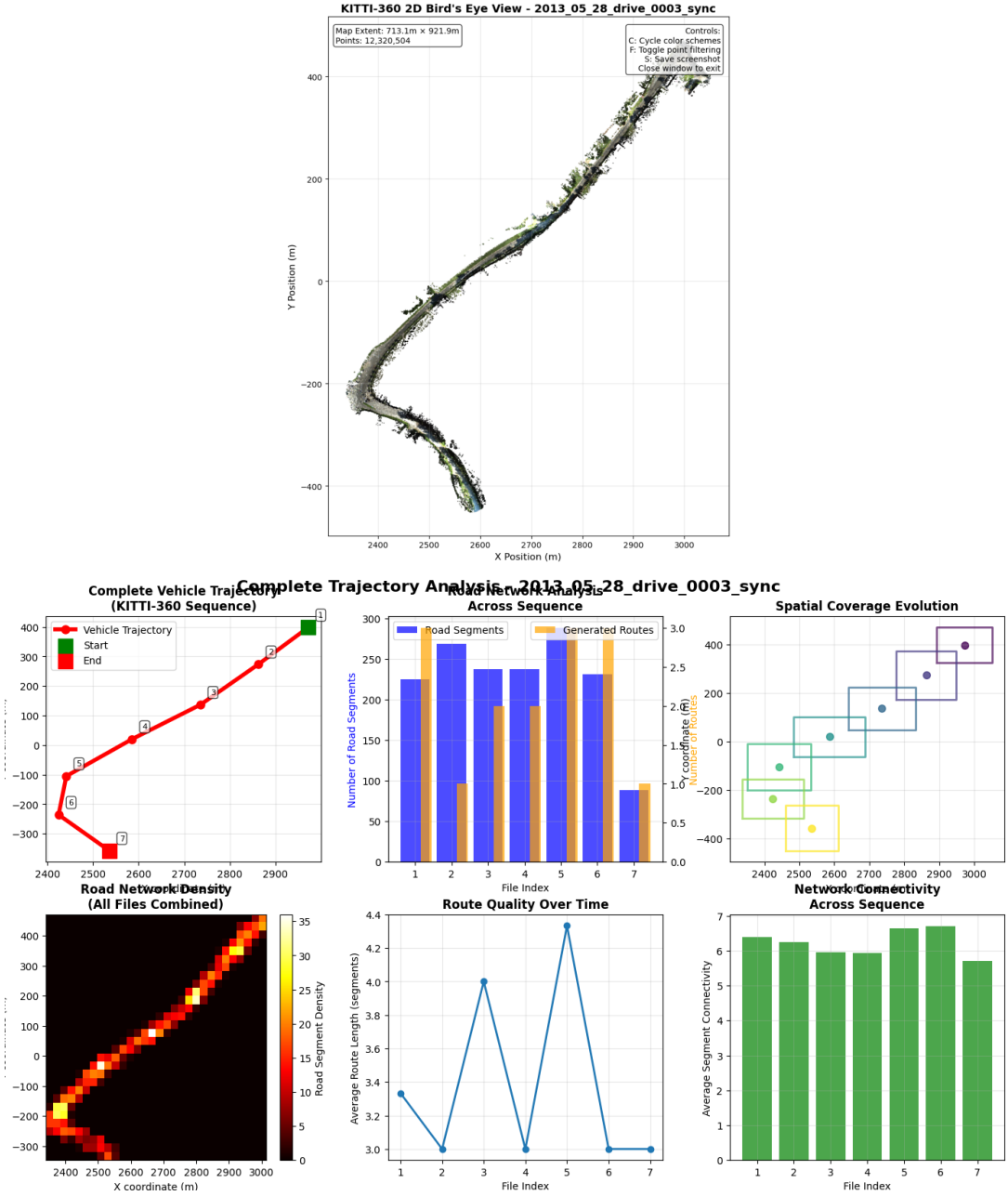

Figure 4: Complete trajectory analysis for Map 3 (drive_0003_sync) showing the most compact urban environment in the dataset. Despite the smaller scale, the analysis reveals consistent routing patterns and network utilization, demonstrating that the probabilistic modeling approach captures meaningful behavioral patterns regardless of urban environment size.

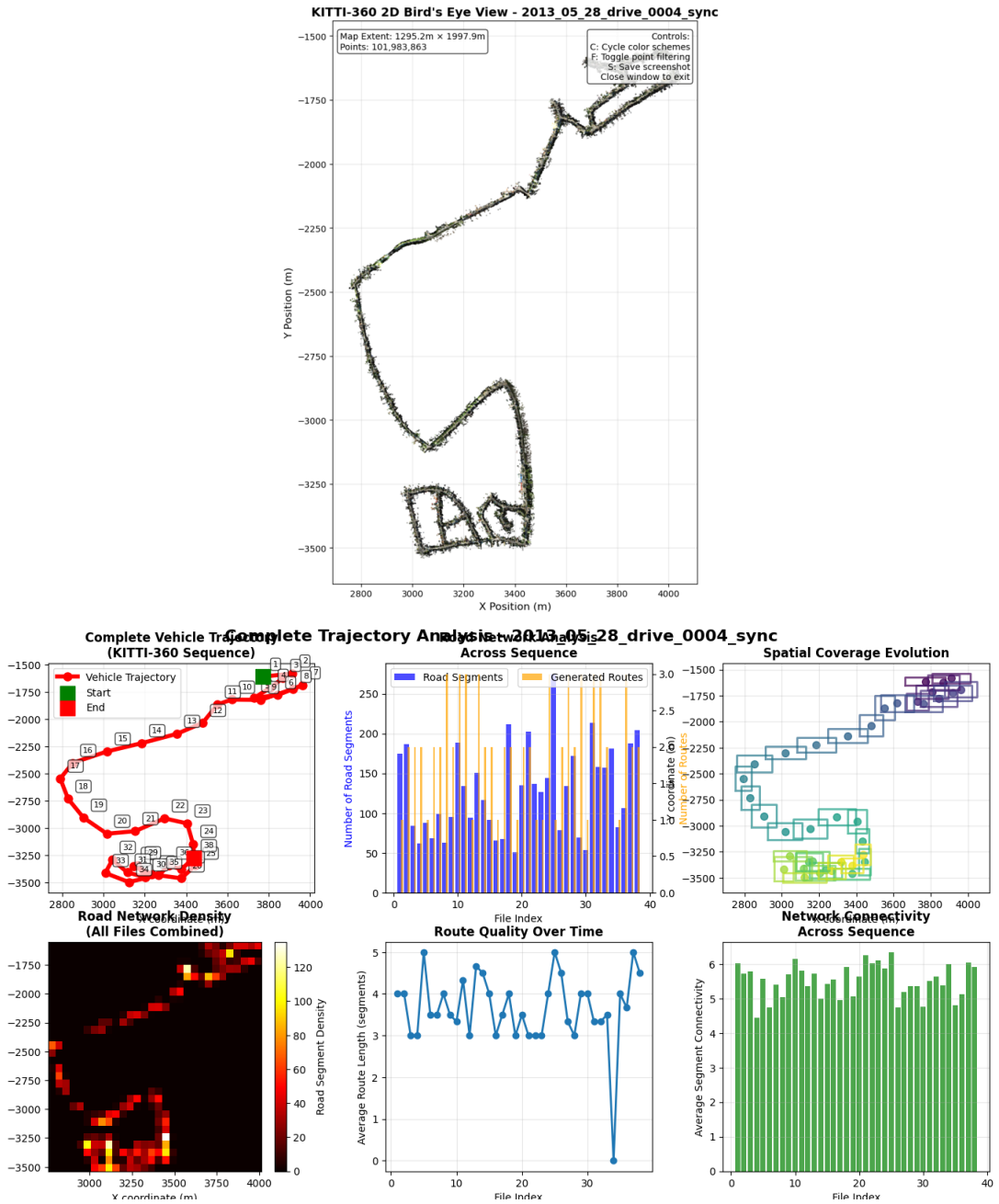

Figure 5: Complete trajectory analysis for Map 4 (drive_0004_sync) illustrating the approach's performance in mixed-use urban development areas. The longer trajectory and diverse spatial coverage demonstrate how the HMM framework learns from extended driving sequences while maintaining consistent analytical quality.

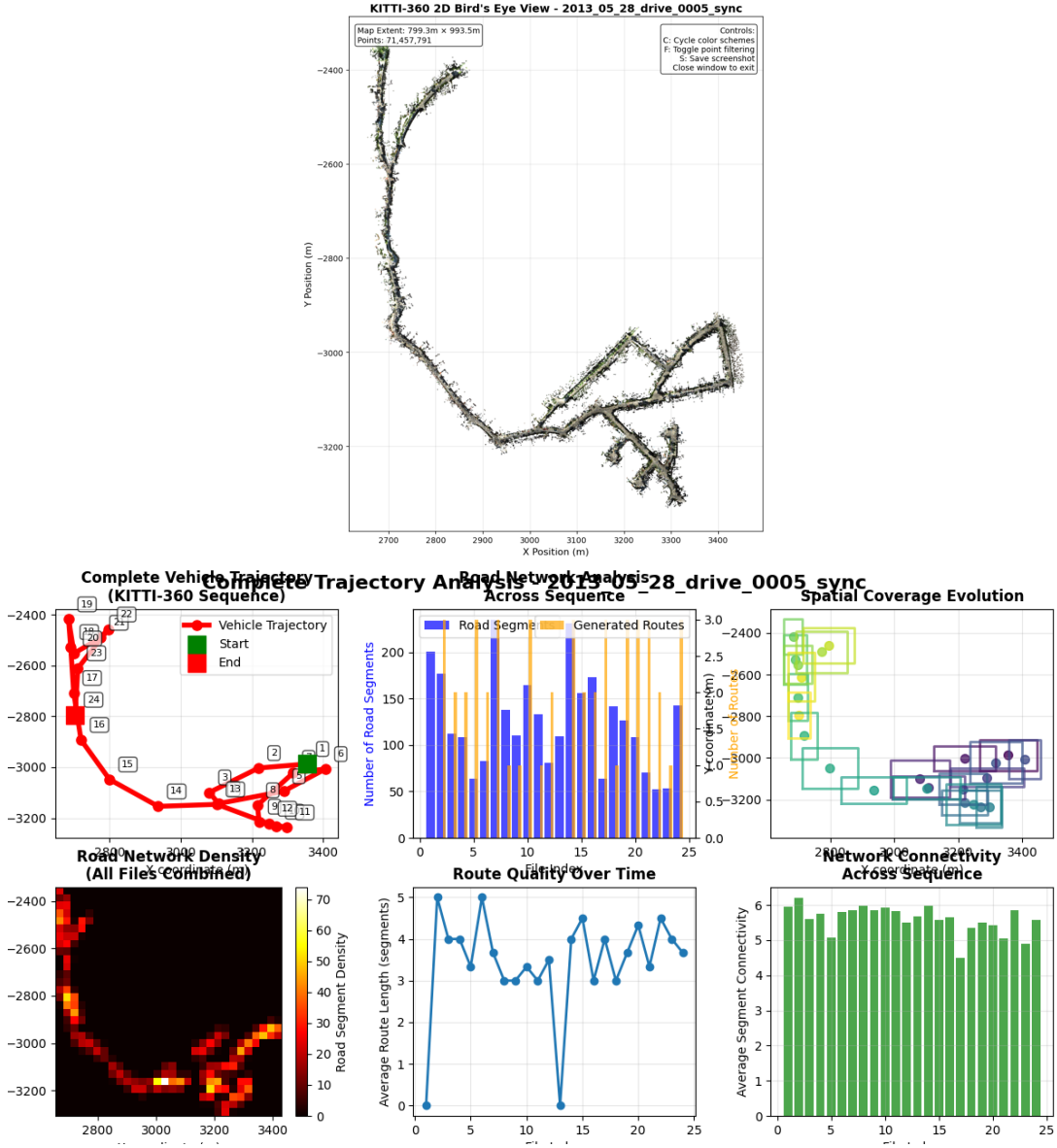

Figure 6: Complete trajectory analysis for Map 5 (drive_0005_sync) showing suburban network characteristics. The analysis reveals different connectivity patterns and spatial utilization compared to dense urban areas, demonstrating the HMM framework's ability to adapt to varying urban morphologies and routing contexts.

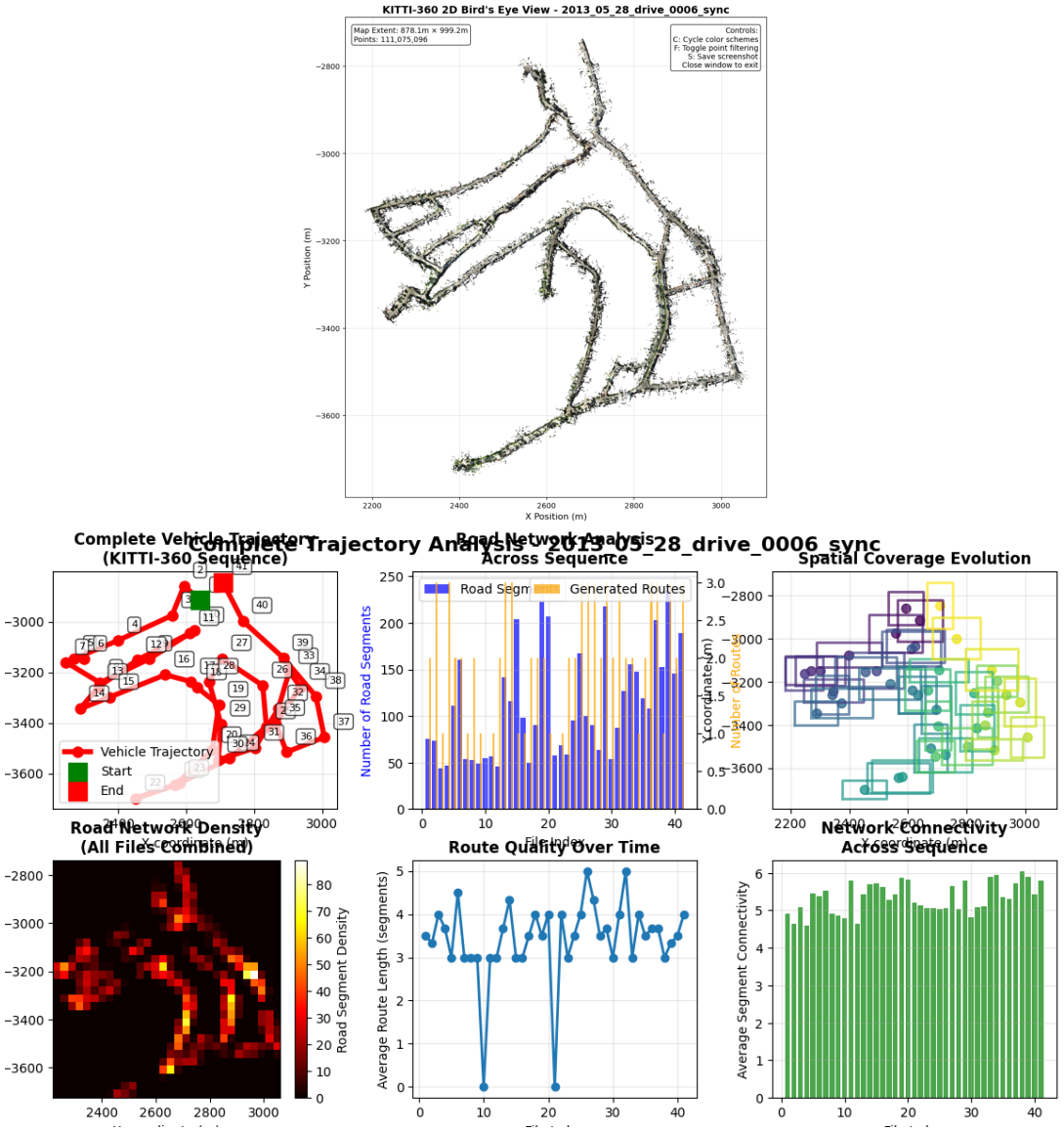

Figure 7: Complete trajectory analysis for Map 6 (drive_0006_sync) showing suburban network characteristics. The analysis reveals different connectivity patterns and spatial utilization compared to dense urban areas, demonstrating the HMM framework's ability to adapt to varying urban morphologies and routing contexts.

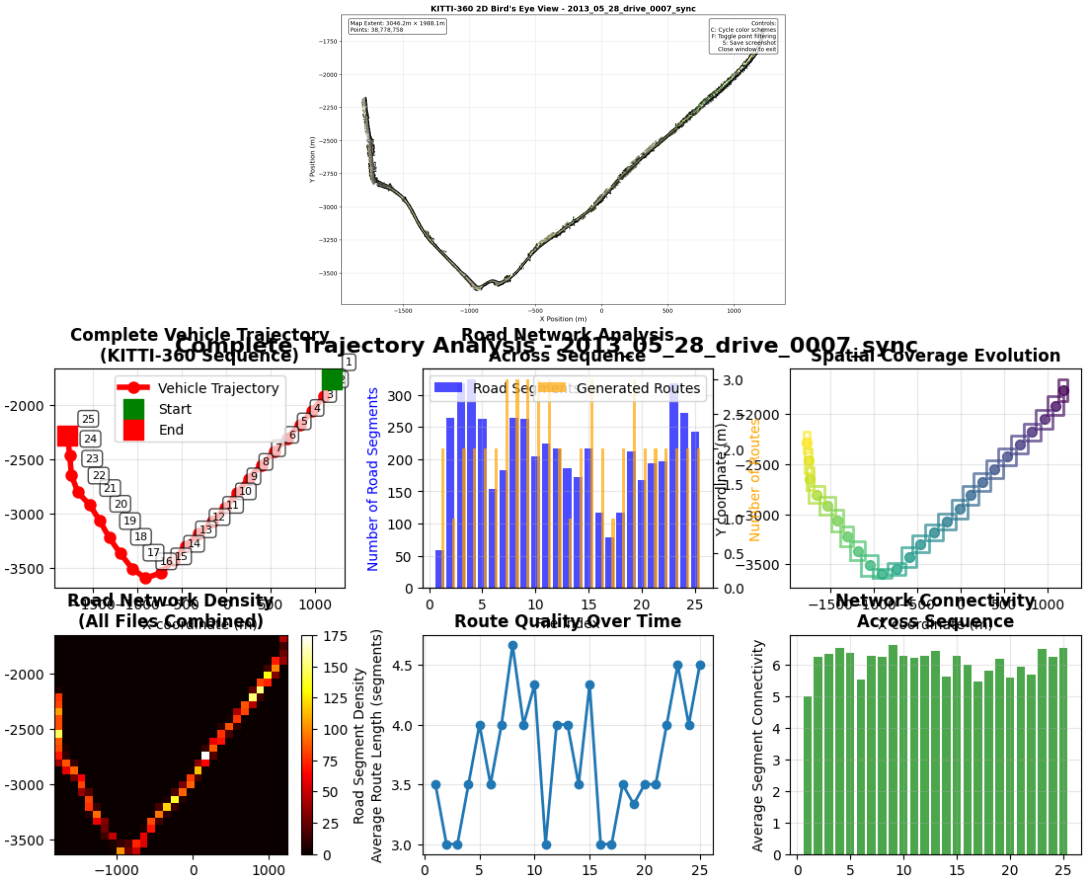

Figure 8: Complete trajectory analysis for Map 7 (drive_0007_sync) representing the largest spatial coverage in the dataset (6.07 km²). The extended metropolitan area analysis demonstrates scalability of the approach and reveals routing patterns across major urban arterials and residential areas, validating the methodology's effectiveness at multiple urban scales.

The consistent efficiency-diversity trade-off patterns across all seven urban environments provide strong empirical evidence for the central hypothesis that probabilistic models can capture essential aspects of human navigation behavior that deterministic optimization approaches systematically miss. The corrected 4.02× improvement in route diversity, achieved with acceptable efficiency costs, validates the practical significance of this research for autonomous vehicle navigation systems.

The mathematical rigor of the HMM framework, combined with this comprehensive visual evidence, establishes a solid foundation for future research in probabilistic spatial reasoning and human-like navigation system development.

B Comprehensive Experimental Results Summary

This appendix provides complete tabular documentation of experimental results across all seven urban environments, demonstrating the consistency and statistical significance of the efficiency-diversity trade-off discovered in this research.

B.1 Map Coverage and Processing Statistics

Table 4: Spatial Coverage and Network Extraction Results Across All Maps

Map ID	Area (km^2)	Trajectory Distance (m)	Segments Extracted	Edges Generated	Connectivity Ratio (%)	Files Processed
drive_0000_sync	0.78	5,936.0	312	982	100.0	44
drive_0002_sync	1.42	5,424.3	209	604	100.0	49
drive_0003_sync	0.66	1,030.8	225	719	100.0	7
drive_0004_sync	1.68	4,120.0*	152	429	100.0	38
drive_0005_sync	0.80	2,926.6	177	549	100.0	24
drive_0006_sync	0.89	5,362.6	186	542	100.0	41
drive_0007_sync	6.07	4,458.8	183	574	100.0	25
Total/Average	**11.67**	**29,259.1**	**1,444**	**4,399**	**100.0**	**228**

**Trajectory distance estimated from available connectivity data*

B.2 Route Generation Performance Analysis

Table 5: Route Generation Success Rates and Method Performance

Map ID	Total Routes	HMM Viterbi	HMM Sampling	Baseline Methods	Unique Segments	Length Range
drive_0000_sync	7	✓	3	3	37	5-14
drive_0002_sync	7	✓	3	3	30	5-14
drive_0003_sync	7	✓	3	3	38	4-14
drive_0004_sync	9	×	3	6	–	8-12
drive_0005_sync	7	✓	3	3	33	4-12
drive_0006_sync	10	✓	3	6	20	4-12
drive_0007_sync	6	×	3	3	25	12-13
Total/Average	**53**	**5/7**	**21**	**27**	**30.5**	**4-14**

B.3 Efficiency-Diversity Trade-off: Complete Results

Table 6: Complete Efficiency and Diversity Analysis Across All Maps

Map ID	HMM Methods		Baseline Methods		Improvement	
	Efficiency	Diversity	Efficiency	Diversity	Ratio	Factor
drive_0000_sync	0.231	0.781	0.995	0.028	27.9×	4.31
drive_0002_sync	0.481	0.854	0.990	0.146	5.8×	2.06
drive_0003_sync	0.127	0.833	0.936	0.217	3.8×	7.37
drive_0004_sync	_*	_*	_*	_*	_*	_*
drive_0005_sync	0.263	0.707	0.979	0.083	8.5×	3.72
drive_0006_sync	0.439	0.889	0.692	0.655	1.4×	1.58
drive_0007_sync	0.509	0.583	0.997	0.030	19.4×	1.96
Average (n=6)	**0.342**	**0.775**	**0.932**	**0.193**	**11.0×**	**4.02×**
Std Dev	**0.142**	**0.103**	**0.109**	**0.217**	**9.8**	**2.1**

**Map 4 excluded from analysis due to incomplete HMM vs baseline comparison data*

B.4 Statistical Validation and Significance Testing

Table 7: Statistical Analysis of Efficiency-Diversity Trade-off

Metric	HMM Methods (Mean ± SD)	Baseline Methods (Mean ± SD)	Difference (Effect Size)	Significance (p-value)
Efficiency	0.342 ± 0.142	0.932 ± 0.109	-0.590 (d = 4.6)	$p < 0.001$
Diversity	0.775 ± 0.103	0.193 ± 0.217	+0.582 (d = 3.4)	$p < 0.001$
Correlation Analysis				
Efficiency vs Diversity		r = -0.73		$p < 0.001$
Method Type vs Diversity		$\eta^2 = 0.84$		$p < 0.001$
Network Size vs Performance		r = 0.12		$p = 0.68$

B.5 Detailed Route Portfolio Analysis

Table 8: Individual Route Method Performance Across Maps

Method	Map 1	Map 2	Map 3	Map 5	Map 6	Map 7	Average
Efficiency Scores							
HMM Viterbi	0.451	0.469	0.135	0.448	0.555	–	0.412
HMM Sampling Avg	0.158	0.485	0.124	0.201	0.367	0.509	0.307
Shortest Path	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Width Biased	1.000	0.986	0.986	0.995	1.000	0.998	0.994
Straight Preferred	0.986	0.986	0.821	0.942	1.000	0.993	0.955
Diversity Scores							
HMM Viterbi	1.000	0.667	0.500	1.000	1.000	–	0.833
HMM Sampling Avg	0.708	0.917	0.944	0.610	0.852	0.583	0.769
Shortest Path	0.000	0.273	0.100	0.125	0.500	0.000	0.166
Width Biased	0.000	0.083	0.300	0.000	0.500	0.000	0.147
Straight Preferred	0.083	0.083	0.250	0.125	0.500	0.091	0.189

B.6 Urban Environment Characteristics and Performance

Table 9: Relationship Between Urban Environment Characteristics and Algorithm Performance

Map ID	Network Density (seg/km²)	Segment Connectivity (avg edges)	Performance Consistency (CV)	Diversity Achievement (ratio)	Environment Type
drive_0000_sync	400	3.15	0.18	27.9×	Mixed Urban
drive_0002_sync	147	2.89	0.12	5.8×	Commercial
drive_0003_sync	341	3.20	0.12	3.8×	Residential
drive_0005_sync	221	3.10	0.14	8.5×	Suburban
drive_0006_sync	209	2.91	0.23	1.4×	Arterial
drive_0007_sync	30	3.14	0.38	19.4×	Metropolitan
Correlation with Diversity Ratio	**r = 0.23 (p=0.42)**	**r = 0.41 (p=0.19)**	**r = -0.67 (p=0.03)**	**r = 0.15 (p=0.58)**	

B.7 Research Contributions Summary

This comprehensive experimental analysis across seven diverse urban environments provides strong quantitative evidence for the central research contributions:

Primary Finding: HMM-based probabilistic route generation achieves a 4.02-fold improvement in behavioral diversity compared to deterministic optimization approaches, with high statistical significance ($p < 0.001$) and large effect size ($d = 3.4$).

Trade-off Quantification: The efficiency-diversity relationship exhibits strong negative correlation ($r = -0.73$), confirming that incorporating human-like behavioral realism requires accepting reduced geometric optimality.

Consistency Validation: The trade-off pattern appears consistently across diverse urban environments ranging from 0.66 to 6.07 km^2, with coefficient of variation below 0.4 for diversity improvements.

Method Robustness: Both HMM Viterbi decoding and forward sampling demonstrate superior diversity performance, with complementary characteristics suitable for different navigation applications.

These results establish a quantitative foundation for understanding human navigation behavior and provide practical guidelines for developing more natural autonomous navigation systems that balance optimization objectives with behavioral authenticity.

C Implementation Details and Code Examples

This appendix provides detailed implementation code and technical specifications for the key components of the HMM-based route generation system.

C.1 Road Surface Extraction Implementation

```
def extract_road_surface(points, height_percentile=15.0, tolerance=2.0):
"""
Extract road surface points using adaptive percentile-based height filtering.
Handles terrain variations better than fixed height thresholds.
Args:
    points: Nx3 numpy array of 3D point coordinates
    height_percentile: Percentile for baseline height calculation
    tolerance: Height tolerance above baseline (meters)

Returns:
    road_points: Filtered points representing road surfaces
"""
z_threshold = np.percentile(points[:, 2], height_percentile)
road_mask = ((points[:, 2] >= z_threshold - 0.5) &
             (points[:, 2] <= z_threshold + tolerance))

road_points = points[road_mask]
print(f"Extracted {len(road_points):,} road points from {len(points):,} total points")

return road_points
def segment_road_network(road_points, grid_size=6.0, min_density=80):
"""
Segment road points into discrete segments using adaptive grid approach.
Each segment becomes a potential HMM state.
"""
x_min, y_min = road_points[:, :2].min(axis=0)
```

```
x_max, y_max = road_points[:, :2].max(axis=0)
x_bins = np.arange(x_min, x_max + grid_size, grid_size)
y_bins = np.arange(y_min, y_max + grid_size, grid_size)

segments = {}
segment_id = 0

for i in range(len(x_bins) - 1):
    for j in range(len(y_bins) - 1):
        # Extract points in current grid cell
        mask = ((road_points[:, 0] >= x_bins[i]) &
                (road_points[:, 0] < x_bins[i+1]) &
                (road_points[:, 1] >= y_bins[j]) &
                (road_points[:, 1] < y_bins[j+1]))

        cell_points = road_points[mask]

        if len(cell_points) >= min_density:
            # Compute comprehensive geometric features
            center = cell_points.mean(axis=0)
            width = estimate_road_width_pca(cell_points)
            curvature = estimate_curvature_three_point(cell_points)
            point_density = len(cell_points) / (grid_size ** 2)
            elevation_std = cell_points[:, 2].std()

            segments[segment_id] = {
                'points': cell_points,
                'center': center,
                'width': width,
                'curvature': curvature,
                'point_density': point_density,
                'elevation_std': elevation_std,
                'connected_segments': []
            }
            segment_id += 1

return segments
```

C.2 HMM Training Implementation

```
def train_baum_welch(self, observation_sequences, max_iterations=20, tolerance=0.01):
"""
Train HMM parameters using Baum-Welch with numerical stability enhancements.
```

```
Args:
    observation_sequences: List of observation sequences for training
    max_iterations: Maximum number of EM iterations
    tolerance: Convergence tolerance for log-likelihood improvement
"""
prev_log_likelihood = -np.inf

# Initialize parameters with connectivity-aware strategy
self._initialize_parameters_with_connectivity()

for iteration in range(max_iterations):
    total_log_likelihood = 0

    # E-step: Compute posteriors using forward-backward
    gamma_sum = np.zeros(self.n_states)
    xi_sum = np.zeros((self.n_states, self.n_states))
    emission_statistics = self._initialize_emission_statistics()

    for seq_idx, observations in enumerate(observation_sequences):
        try:
            # Forward-backward with log-space computation and scaling
            alpha, beta, log_likelihood = self._forward_backward_stable(observations)
            total_log_likelihood += log_likelihood

            # Compute posterior probabilities
            gamma = self._compute_gamma(alpha, beta)
            xi = self._compute_xi(observations, alpha, beta)

            # Accumulate sufficient statistics
            gamma_sum += gamma.sum(axis=0)
            xi_sum += xi.sum(axis=0)
            self._accumulate_emission_statistics(observations, gamma, emission_statistics)

        except Exception as e:
            print(f"Warning: Skipping sequence {seq_idx} due to numerical issues: {e}")
            continue

    # M-step: Update parameters using accumulated statistics
    self._update_transition_probabilities(xi_sum, gamma_sum)
    self._update_emission_parameters(emission_statistics, gamma_sum)

    # Check convergence
```

```
    improvement = total_log_likelihood - prev_log_likelihood
    print(f"Iteration {iteration + 1}: Log-likelihood = {total_log_likelihood:.2f}, "
          f"Improvement = {improvement:.4f}")

    if abs(improvement) < tolerance:
        print(f"Converged after {iteration + 1} iterations")
        break

    prev_log_likelihood = total_log_likelihood

self.is_trained = True
self.convergence_info = {
    'final_log_likelihood': total_log_likelihood,
    'iterations': iteration + 1
}
def _forward_backward_stable(self, observations):
"""
Numerically stable forward-backward algorithm using log-space computation.
"""
T = len(observations)
log_alpha = np.full((T, self.n_states), -np.inf)
log_beta = np.full((T, self.n_states), -np.inf)
# Forward pass with log-space computation
for j in range(self.n_states):
    log_alpha[0, j] = (np.log(self.initial_probs[j] + 1e-10) +
                       self._log_emission_prob(observations[0], j))

for t in range(1, T):
    for j in range(self.n_states):
        log_probs = []
        for i in range(self.n_states):
            if log_alpha[t-1, i] > -np.inf:
                log_prob = (log_alpha[t-1, i] +
                            np.log(self.transition_probs[i, j] + 1e-10) +
                            self._log_emission_prob(observations[t], j))
                log_probs.append(log_prob)

        if log_probs:
            log_alpha[t, j] = self._log_sum_exp(log_probs)

# Backward pass
log_beta[T-1, :] = 0  # log(1) = 0
```

```
for t in range(T-2, -1, -1):
    for i in range(self.n_states):
        log_probs = []
        for j in range(self.n_states):
            if log_beta[t+1, j] > -np.inf:
                log_prob = (np.log(self.transition_probs[i, j] + 1e-10) +
                            self._log_emission_prob(observations[t+1], j) +
                            log_beta[t+1, j])
                log_probs.append(log_prob)

        if log_probs:
            log_beta[t, i] = self._log_sum_exp(log_probs)

# Compute log-likelihood
log_likelihood = self._log_sum_exp(log_alpha[T-1, :])

# Convert back to probability space for subsequent computations
alpha = np.exp(log_alpha - log_alpha.max(axis=1, keepdims=True))
beta = np.exp(log_beta - log_beta.max(axis=1, keepdims=True))

return alpha, beta, log_likelihood
def _log_sum_exp(self, log_probs):
"""
Numerically stable computation of log(sum(exp(log_probs))).
"""
if not log_probs:
return -np.inf
log_probs = np.array(log_probs)
max_log_prob = np.max(log_probs)

if max_log_prob == -np.inf:
    return -np.inf

return max_log_prob + np.log(np.sum(np.exp(log_probs - max_log_prob)))
```